Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 7

AIM:
To implement secure social authentication using Better Auth with Google and GitHub OAuth providers on a Next.js platform.

SOFTWARE REQUIRED:
- **Operating System:** Linux / macOS / Windows 10/11
- **Database Engine:** PostgreSQL (running via Docker Compose or local instance)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code
- **Developer Accounts:** Google Cloud Console & GitHub Developer Settings

THEORY:
- **Modern Authentication with Better Auth:** Better Auth is a TypeScript-first, framework-agnostic authentication framework tailored for Next.js App Router. It abstracts session handling, OAuth 2.0/OIDC handshakes, CSRF mitigation, and database persistence into a modular server configuration with typed client libraries.
- **OAuth 2.0 Authorization Code Flow:** Social login follows the OAuth 2.0 Authorization Code Grant:
    1. The client initiates login, redirecting the user's browser to the provider (Google/GitHub) with client_id, requested scope, and a cryptographic state token to prevent CSRF attacks.
    2. The user authenticates and approves basic profile permissions (openid, profile, email).
    3. The provider redirects back to the configured application callback URL (/api/auth/callback/[provider]) with an authorization code.
    4. Better Auth exchanges the authorization code on the server for an access_token and identity payload, creates or links database records (user, account, session), and issues a secure, HTTP-only session cookie.
- **Database Persistence Strategy:** Better Auth maintains normalized tables to handle identity lifecycles:
    - user: Stores canonical user identities (id, name, email, avatar).
    - session: Stores active sessions with expiry timestamps and associated token hashes.
    - account: Maps third-party OAuth provider identifiers (google, github) and provider account IDs to internal user IDs.
    - verification: Stores transient tokens for email verification and one-time password handshakes.

- **App Router Catch-All Integration (**toNextJsHandler**):** Next.js App Router intercepts authentication actions via an all-inclusive catch-all route handler (app/api/auth/[...all]/route.ts). Better Auth translates internal HTTP router requests into standard Next.js Web Request and Response interfaces.

STEPS / ALGORITHM:

Step 1: Install Dependencies

Install Better Auth core, client bindings, and the PostgreSQL driver adapter:

```
npm install better-auth pg
npm install @types/pg --save-dev
```

Step 2: Configure OAuth Credentials & Environment (.env**)**

Configure your database connection, secret keys, and provider credentials in .env:

```
BETTER_AUTH_SECRET="your-32-character-random-secret-key-here"
BETTER_AUTH_URL="http://localhost:3000"

DATABASE_URL="postgresql://devuser:devpassword@localhost:5432
/nextjs_blog_db?schema=public"

# Google Cloud Console OAuth Credentials
GOOGLE_CLIENT_ID="your-google-client-
id.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="your-google-client-secret"

# GitHub Developer Settings OAuth Credentials
GITHUB_CLIENT_ID="your-github-client-id"
GITHUB_CLIENT_SECRET="your-github-client-secret"
```

Important OAuth Redirect Configuration:

- **Google Authorized Redirect URI:** http://localhost:3000/api/auth/callback/google
- **GitHub Authorization Callback URL:** http://localhost:3000/api/auth/callback/github

Step 3: Configure Server-Side Auth Instance (app/lib/auth.ts**)**

Initialize the Better Auth configuration using a connection pool for PostgreSQL:

```
import { betterAuth } from 'better-auth';
import { Pool } from 'pg';

export const auth = betterAuth({
  database: new Pool({
```

```
    connectionString: process.env.DATABASE_URL,
  }),
  baseURL: process.env.BETTER_AUTH_URL ||
'http://localhost:3000',
  socialProviders: {
    github: {
      clientId: process.env.GITHUB_CLIENT_ID as string,
      clientSecret: process.env.GITHUB_CLIENT_SECRET as
string,
    },
    google: {
      clientId: process.env.GOOGLE_CLIENT_ID as string,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET as
string,
    },
  },
});
```

Step 4: Provision Auth Database Schema via CLI

Execute the Better Auth CLI migration pointing directly to your auth configuration file:

```
npx @better-auth/cli migrate --config app/lib/auth.ts
```

This creates the user, session, account, and verification tables inside your PostgreSQL database.

Step 5: Mount Next.js Catch-All Route Handler (app/api/auth/[...all]/route.ts**)**

Create the dynamic catch-all route to expose Better Auth endpoints:

```
import { auth } from '@/app/lib/auth';
import { toNextJsHandler } from 'better-auth/next-js';

export const { GET, POST } = toNextJsHandler(auth);
```

Step 6: Create Typed Client Instance (app/lib/auth-client.ts**)**

Instantiate the client authentication helper without hardcoded host URLs so it dynamically resolves to the browser's current origin:

```
import { createAuthClient } from 'better-auth/react';

export const authClient = createAuthClient();

export const { signIn, signOut, useSession } = authClient;
```

Step 7: Build Authentication & Profile UI View (app/auth-demo/page.tsx**)**

Create a responsive client interface to execute social authentication handshakes and render session details:

```
'use client';

import { signIn, signOut, useSession } from '@/app/lib/auth-
client';

export default function AuthDemoPage() {
  const { data: session, isPending } = useSession();

  const handleSignIn = async (provider: 'google' | 'github')
=> {
    await signIn.social({
      provider,
      callbackURL: '/auth-demo',
    });
  };

  const handleSignOut = async () => {
    await signOut({
      fetchOptions: {
        onSuccess: () => {
          window.location.reload();
        },
      },
    });
  };

  return (
    <div className="min-h-screen bg-slate-50 text-slate-900
p-6 sm:p-10 font-sans">
      <div className="max-w-xl mx-auto space-y-8">

        {/* Header */}
        <header className="border-b border-slate-200 pb-5">
          <div className="flex items-center gap-2 mb-2">
            <span className="px-2.5 py-0.5 rounded-full text-
xs font-semibold bg-indigo-100 text-indigo-700">
              Experiment 7
            </span>
            <span className="text-xs text-slate-500 font-
mono">
              Next.js 16 + Better Auth
```

```
            </span>
          </div>
          <h1 className="text-2xl sm:text-3xl font-bold
tracking-tight text-slate-900">
            Social Authentication
          </h1>
          <p className="mt-1 text-sm text-slate-600">
            Secure OAuth 2.0 authentication using Google and
GitHub identity providers.
          </p>
        </header>

        {/* State / Interactive Authentication Area */}
        <section className="bg-white rounded-xl border
border-slate-200 shadow-sm p-6 sm:p-8">
          {isPending ? (
            <div className="text-center py-6 text-slate-500
text-sm">
              Verifying active session...
            </div>
          ) : session ? (
            /* Authenticated User Profile View */
            <div className="space-y-6">
              <div className="flex items-center gap-4">
                {session.user.image ? (
                  <img
                    src={session.user.image}
                    alt={session.user.name || 'User Avatar'}
                    className="w-14 h-14 rounded-full border
border-slate-200 object-cover"
                  />
                ) : (
                  <div className="w-14 h-14 rounded-full bg-
slate-200 flex items-center justify-center text-slate-600
font-bold">
                    {session.user.name?.charAt(0) || 'U'}
                  </div>
                )}
                <div>
                  <h2 className="text-lg font-semibold text-
slate-900">
```

```
                    {session.user.name}
                  </h2>
                  <p className="text-sm text-slate-
500">{session.user.email}</p>
                </div>
              </div>

              <div className="p-4 bg-slate-50 rounded-lg
border border-slate-200 text-xs font-mono space-y-1">
                <p><strong>User ID:</strong>
{session.user.id}</p>
                <p><strong>Session Expires:</strong> {new
Date(session.session.expiresAt).toLocaleString()}</p>
              </div>

              <button
                onClick={handleSignOut}
                className="w-full py-2.5 bg-rose-600
hover:bg-rose-700 text-white text-sm font-semibold rounded-lg
shadow transition cursor-pointer"
              >
                Sign Out
              </button>
            </div>
          ) : (
            /* Unauthenticated Action View */
            <div className="space-y-4">
              <h2 className="text-lg font-semibold text-
slate-900 mb-1">
                Sign In to Your Account
              </h2>
              <p className="text-xs text-slate-500 mb-4">
                Select an OAuth provider to initiate identity
verification.
              </p>

              <button
                onClick={() => handleSignIn('github')}
                className="w-full flex items-center justify-
center gap-2 py-2.5 bg-slate-900 hover:bg-slate-800 text-
```

```
white text-sm font-semibold rounded-lg shadow transition
cursor-pointer"
              >
                Continue with GitHub
              </button>

              <button
                onClick={() => handleSignIn('google')}
                className="w-full flex items-center justify-
center gap-2 py-2.5 bg-white hover:bg-slate-50 text-slate-700
border border-slate-300 text-sm font-semibold rounded-lg
shadow-sm transition cursor-pointer"
              >
                Continue with Google
              </button>
            </div>
          )}
        </section>

      </div>
    </div>
  );
}
```

Step 8: Validate the Authentication Lifecycle

1. Execute the Next.js development server:

```
npm run dev
```

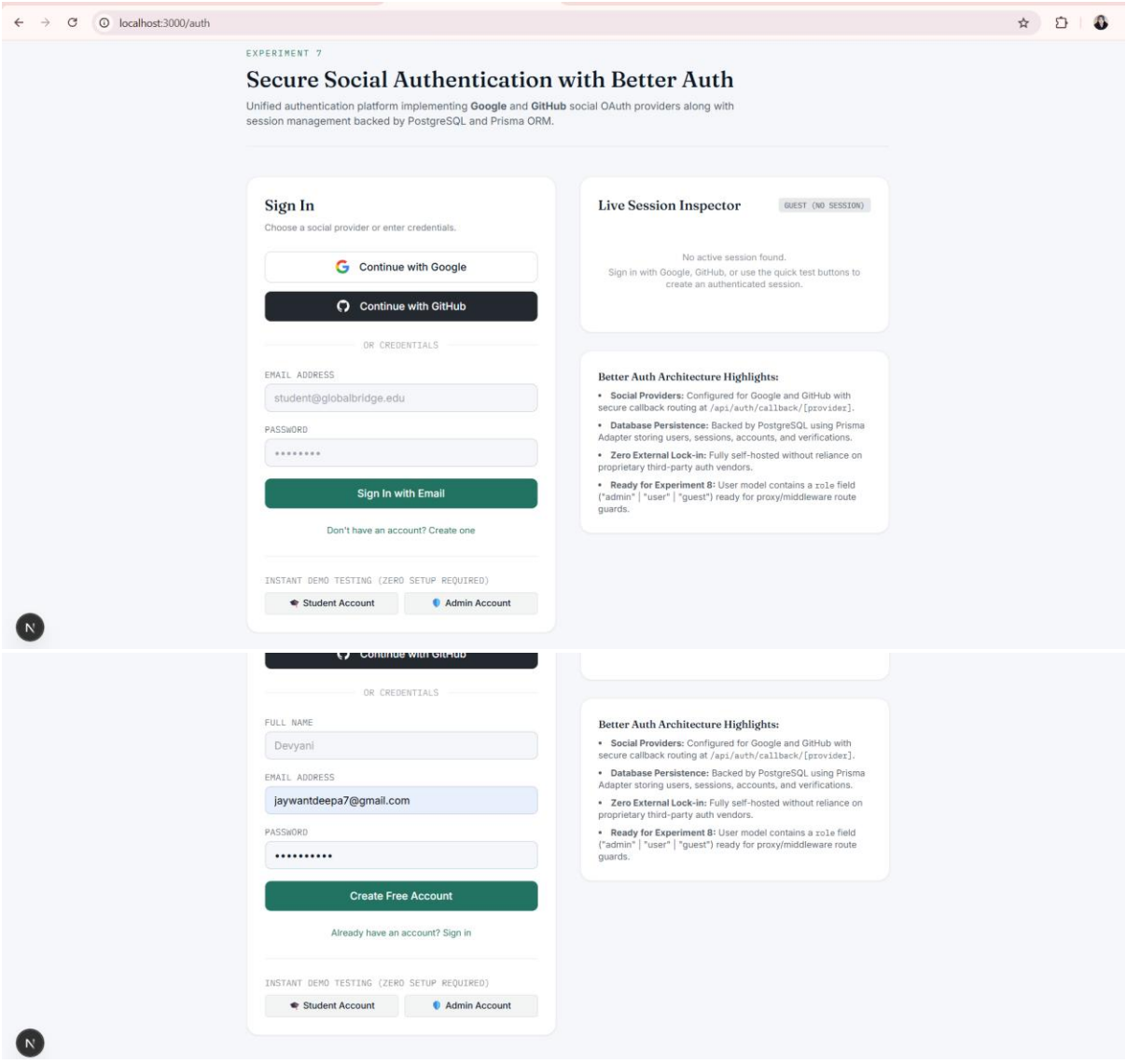
2. Navigate to http://localhost:3000/auth-demo.
3. Click **Continue with GitHub** or **Continue with Google**.
4. Authorize the application on the consent screen.
5. Verify redirection back to /auth-demo, confirming that user profile details (name, email, avatar image) render alongside a valid session token.

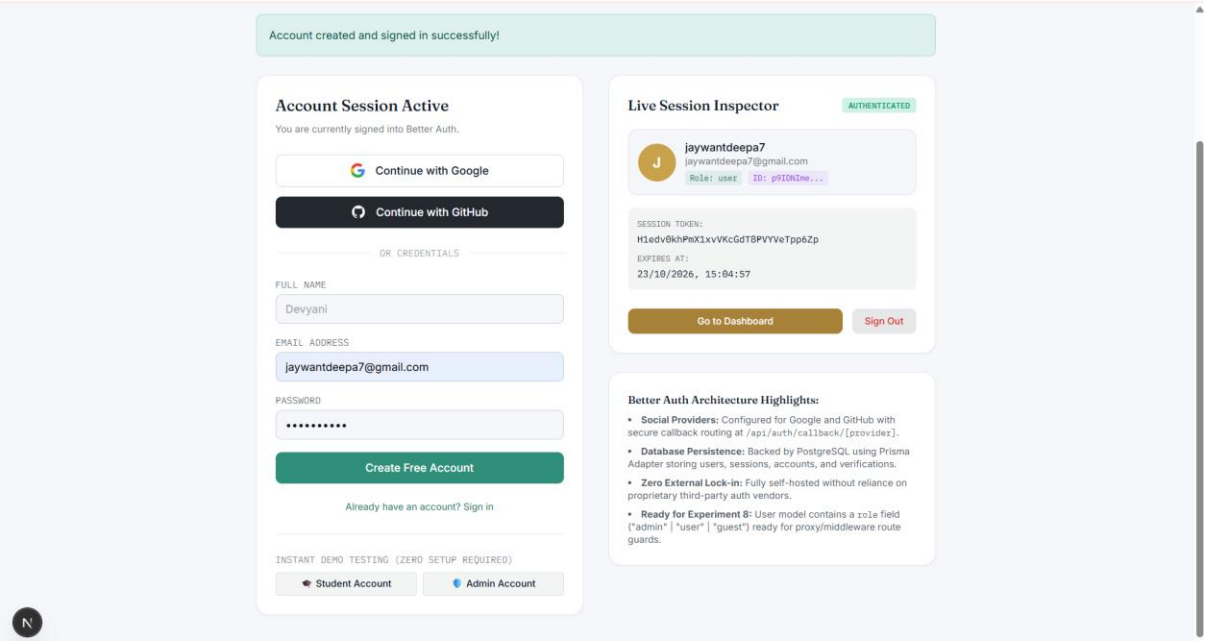
OBSERVATION:

Observe how selecting an identity provider triggers an immediate redirection to the OAuth provider's authorization gateway. When consent is granted, the provider returns an authorization code via /api/auth/callback/[provider].

Better Auth intercepts this code, exchanges it for user claims on the server side, and writes records into the PostgreSQL user and account tables.

Open browser Developer Tools (Application $\rightarrow$ Storage $\rightarrow$ Cookies) to verify the encrypted, HTTP-only cookie (better-auth.session_token). Refreshing the browser preserves the active session state without triggering re-authentication prompts.





CONCLUSION:

Secure social authentication using Better Auth with Google and GitHub OAuth providers was successfully implemented on Next.js. The experiment verified OAuth 2.0 authorization code handshakes, secure server-side session management using PostgreSQL, and client-side session state handling using typed hooks.

POST LAB QUESTION:

Q: Why are OAuth 2.0 client secrets stored strictly in server-side environment variables rather than exposed to the client-side Next.js bundle (NEXT_PUBLIC_*), **and what attack vector does this prevent?**

A:

OAuth client secrets are used by the backend application to authenticate itself against the identity provider when exchanging a temporary authorization code for user access tokens.

If client secrets are exposed to the client bundle (e.g., via NEXT_PUBLIC_*), malicious actors can extract the secret and impersonate the application. An attacker could forge token exchange requests, intercept sensitive profile tokens, or register unauthorized redirect URIs, completely compromising user authentication and enabling account takeover attacks. Keeping credentials strictly server-side ensures the token-exchange step remains confidential.